An Efficient Approach to Apply the RISC-V Sail Model to Chip Verification

Abstract

The Sail RISC-V Model can generate an executable file from its formal specification,. Currrently RISC-V tests only provides limited test cases and cannot comprehensively test your RISC-V implementation. Some chips may use self-developed simulators for testing, but they cannot obtain formal verification-based guarantees like RISC-V Sail Model, nor can they offer full configurability. This work introduces a new test framework that uses the RISC-V Sail Model as the ref model, ensuring the model's completeness and accuracy. To improve simulation performance, we choose to use Pydrofoil, which is an improved version of the Sail Model that delivers ultra-high performance. To enhance test compatibility and usability, we provide a set of simple test interfaces (including register access, memory access, etc.) and support customizing model configurations. Currently, it has successfully integrated tests for some open-source RISC-V implementations.

Introduction

The Sail RISC-V model[1] can generate an executable simulator from a formal specification, and it has already been used in riscv-arch-test (ACT). However, ACT provides only limited test cases and cannot comprehensively validate a full RISC-V implementation in chip-level verification scenarios. In many projects, teams also rely on self-developed simulators to increase testing throughput, but these simulators do not provide the same formal-verification-based confidence as Sail-RISC-V, and they usually offer less complete configurability.

This work starts from that practical gap. We want to introduce the Sail model into broader verification workflows, not only compliance-style tests. The key goal is to keep the model's semantic accuracy while making integration simple enough for real engineering use. We also want the solution to work with existing test frameworks rather than replacing the whole environment.

To achieve this, we use Sail-RISC-V as the reference model and build a practical integration path around it. We choose Pydrofoil[2] to improve execution performance and we provide lightweight interfaces for common verification operations such as register access and memory access. The approach is designed to stay close to existing diff-test flows so that teams can adopt it with limited migration cost.

Methodologies

Our methodology is based on Pydrofoil, which can be seen as a high-performance execution path for Sail-based models. The basic idea is simple: keep the Sail semantic core as the correctness foundation, and use Pydrofoil to make large-scale simulation practical. Pydrofoil applies optimization techniques to the Sail execution process and enables efficient interpretive execution of the model core.

On the integration side, we implement the adapter layer in Python. This allows users to write and maintain testing interfaces quickly, with much less boilerplate than a full C++ or Rust wrapper. At the same time, the Python code can be compiled through PyPy CFFI into native C-level artifacts, so this engineering convenience does not necessarily come with large runtime overhead. In our experience, this balance between development speed and execution efficiency is important for iterative verification work.

We selected Xiangshan[3] as the target processor project. Xiangshan is an open-source high-performance RISC-V implementation written in Chisel, and its codebase includes a difftest framework[4] that traditionally uses NEMU as the reference model. Our direct objective is to replace NEMU's role with a Sail-based dynamic library while preserving the existing difftest workflow as much as possible.

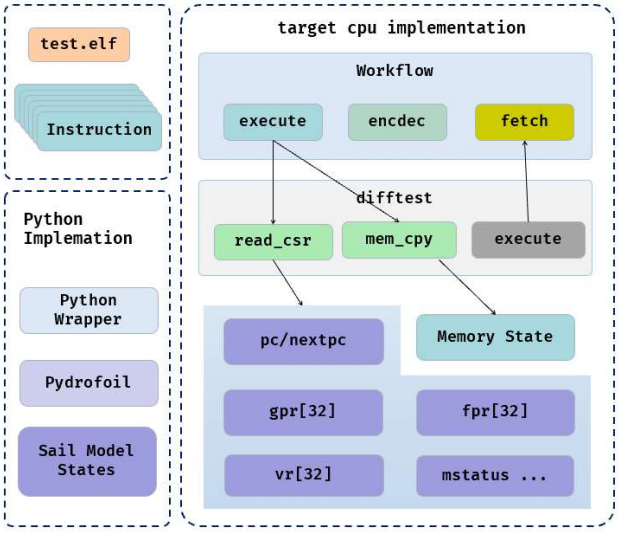

Figure 1 : overview

The difftest framework drives test execution and state comparison. It depends on several key interfaces, including

execution control and state access interfaces such as execute, read_csr, and mem_cpy. Sail naturally provides equivalent model-side capabilities. In the adapter, we map difftest needs to Sail-side operations such as step, rX_bits, and read/write memory calls. This mapping is the core of the integration. Because we keep the interface surface small and concrete, the implementation remains manageable while still supporting practical verification tasks.

Results

The integration process became significantly easier after moving to the Pydrofoil-based path. The core implementation required only about 800 lines of Python code. Most of this code was not about instruction stepping itself; instead, it focused on practical alignment work, especially CSR encoding definitions and register-name mapping between Sail-RISCV and Xiangshan. We also added handling for special state elements such as NextPC, which are necessary for stable state comparison.

From an engineering perspective, this result supports the original motivation. The adapter remained lightweight, the existing difftest workflow could still be used, and the Sail model could be introduced as a formal-semantic reference without a heavy rewrite of the verification infrastructure. In other words, the method achieved a usable tradeoff among correctness, performance, and implementation effort.

At the same time, our experiments also exposed clear limitations. Although Sail-RISCV is highly configurable, in our setup it did not fully match Xiangshan's feature set around RVA23-related behavior. This caused model-state misalignment in differential runs. Another practical issue is that Xiangshan does not currently provide a convenient interface to disable some extension sets, which makes capability alignment harder during experiments. After attempts to disable the H extension, we then encountered PMP-related state issues, and one run stopped at an instruction count of 894,352. These observations are consistent with the original paper's point: semantic correctness of the reference model is necessary, but configuration alignment between DUT and model is equally critical in real deployments.

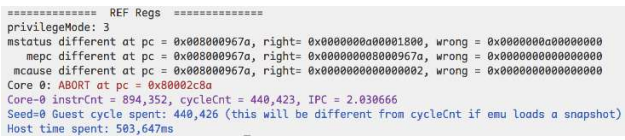

Figure 2 : difftest result

Conclusion

This work presents a practical way to apply the RISC-V Sail model to chip verification by combining Sail semantics, Pydrofoil performance, and a lightweight Python+CFFI adapter. The approach keeps the existing difftest structure largely intact and reduces integration complexity, while preserving the core advantage of a formally grounded reference model. In the Xiangshan case, it was feasible to replace the NEMU reference path with a Sail-based dynamic library using a relatively small codebase.

The experiments also show that future progress depends on toolchain maturity and feature alignment. Pydrofoil is currently tied to a specific commit and still needs substantial updates. Moving to newer versions with broader extension support and more common interfaces is necessary for deeper experimentation and wider adoption. Overall, this approach is not a theoretical proposal only; it is an engineering path that can already be used, and it provides a clear direction for improving formal-model-based verification in practical RISC-V projects.

References

[1] Alasdair Armstrong sail-riscv: Formal Specification of the RISC-V ISA. url: https://github.com/riscv/sail-riscv
[2] Bolz-Tereick, C., Panayi, L., McKeogh, F., Spink, T., & Berger, M. (2025). Pydrofoil: Accelerating Sail-Based Instruction Set Simulators. In *39th European Conference on Object-Oriented Programming (ECOOP 2025)* (pp. 3:1–3:31). Schloss Dagstuhl – Leibniz-Zentrum fur Informatik.
[3] Kunlin You, Yinan Xu, Kehan Feng, Luoshan Cai, Yaoyang Zhou, and Yungang Bao. 2025. DiffTest-H: Toward Semantic-Aware Communication in Hardware-Accelerated Processor Verification. Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture. Association for Computing Machinery, New York, NY, USA,1462–1476. https://doi.org/10.1145/3725843.3756108
[4] K. Wang *et al*., "XiangShan: An Open-Source Project for High-Performance RISC-V Processors Meeting Industrial-Grade Standards," *2024 IEEE Hot Chips 36 Symposium (HCS)*, Stanford, CA, USA, 2024, pp. 1-25, doi: 10.1109/HCS61935.2024.10665293.